

Signals and Systems

Computer Assignment 2

Department of Electrical Engineering
Sharif University of Technology

26 Ordibehesht 1405

Instructor: Dr. Hamid Behroozi

Deadline: As specified on the course system

Contact: If you have any questions, please contact the TA (@reyhane_amz) via Bale or Rubika.

Submission Guidelines

1. Submit your assignment report strictly in **PDF format**. The report must include all outputs, plots, explanations of your solving methods, result analysis, and answers to any theoretical questions.
2. Submit the complete assignment code as a single `.m` file. Different sections of the assignment should be clearly separated using MATLAB sections.
3. Any custom functions you write must be placed at the bottom of the main file in a dedicated section titled **Functions**. Avoid creating multiple separate MATLAB files.
4. Archive all necessary files (including the report, code, and required outputs) into a single `.zip` or `.rar` file and upload it to the course system.
5. Please adhere strictly to the following file naming convention:
 - StudentNumber_CHW2.m
 - StudentNumber_CHW2.pdf
 - StudentNumber_CHW2.zip

Grading Criteria

1. Organization, readability, and completeness of the report.
2. Correct implementation of algorithms and error-free code execution.
3. Proper use of vectorization and efficient computational methods.
4. Quality of plots (including titles, axis labels, legends) and thorough analysis of results.
5. Accurate and detailed answers to analytical and comparative questions.

1. Properties of the Continuous-Time Fourier Transform

In this exercise, you will enhance your understanding of the continuous-time Fourier transform (CTFT) by analyzing and manipulating audio signals in the frequency and time domains. Audio signals in MATLAB are represented by vectors containing samples of the continuous-time audio signal. The sampling rate is assumed to be 8192 Hz, i.e., the audio signal is sampled every $\Delta t = (1/8192)$ seconds. To be more precise, for an audio signal $y(t)$ sampled at 8192 Hz over the interval $0 \leq t < N\Delta t$, the N -element vector $\mathbf{y}$ which represents the audio signal is given by

$$y(n) = y((n-1)\Delta t), \quad n = 1, \dots, N.$$

The function `sound` can then be used to play the signal on your computer's speaker.

While this is a sampled representation of a continuous-time audio signal $y(t)$, if $y(t)$ is zero outside the sampling interval and the sampling rate $f_s = 8192$ Hz is fast enough¹, then $\mathbf{y}$ can be considered an accurate representation of $y(t)$. For all the signals in this exercise, $\mathbf{y}$ can be assumed to be an accurate representation of $y(t)$. To begin this exercise, you must first load a sample audio signal by typing

```
>> load splat
>> y = y(1:8192);
```

To verify that you have accurately loaded the audio data, and to convince yourself that the MATLAB vector $\mathbf{y}$ can accurately represent an audio signal, type

```
>> N = 8192;
>> fs = 8192;
>> sound(y,fs)
```

The function `fft` takes the sampled representation $\mathbf{y}$ and computes an approximation to the CTFT of $y(t)$, $Y(j\omega)$, at samples of ω . If you type

```
>> Y = fftshift(fft(y));
```

then the vector $\mathbf{Y}$ contains an approximation to $Y(j\omega)$ at N evenly spaced frequencies on the interval $-\pi f_s \leq \omega < \pi f_s$. In fact, $\mathbf{Y}$ contains only approximate values of $cY(j\omega)$, where c is a constant. You should not worry about the exact approximation or the scaling for the purposes of this exercise.

The function `fftshift` re-orders the output of `fft` so that the samples of $Y(j\omega)$ are ordered in $\mathbf{Y}$ from the most negative to most positive frequencies. Most of the properties that you associate with the CTFT can now be verified on the vector $\mathbf{Y}$.

- (a). Type `Y=fftshift(fft(y))` to calculate the Fourier transform vector $\mathbf{Y}$. The corresponding frequency values can be stored in the vector $\mathbf{w}$ by typing

```
>> w = [-pi:2*pi/N:pi-pi/N]*fs;
```

Use $\mathbf{w}$ and $\mathbf{Y}$ to plot the magnitude of the continuous-time Fourier transform over the interval $-\pi f_s \leq \omega < \pi f_s$.

¹The Nyquist-Shannon sampling theorem dictates how fast a signal must be sampled so that the sampled version of the signal accurately represents the underlying continuous-time signal.

The function `ifft` is the inverse operation of `fft`. For even length vectors, `fftshift` is its own inverse. For the vector `Y`, $N = 8192$ and the inverse Fourier transform can be found by typing

```
>> y = ifft(fftshift(Y));
>> y = real(y);
```

The `real` function is applied here because the original time-domain signal was known to be real. However, numerical roundoff errors in `fft` and `ifft` can introduce a very small nonzero imaginary component to `y`. In general, the inverse CTFT is not necessarily a real signal, and the imaginary part may contain significant energy. The `real` function should only be used on the output of `ifft` when you know the resulting signal must be real, e.g., an audio signal, and you have verified the imaginary part that you will be removing is insignificant.

- (b). Set `Y1=conj(Y)`, and store in `y1` the inverse Fourier transform of `Y1`. Use `real(y1)` to ensure `y1` is real. Play `y1` using `sound(y1,fs)`. Can you explain what you just heard, knowing how the inverse Fourier transform of $Y^*(j\omega)$ is related to $y(t)$?

The CTFT of $y(t)$ can be written in terms of its magnitude and phase as

$$Y(j\omega) = |Y(j\omega)|e^{j\phi(\omega)},$$

where $\phi(\omega) = \angle Y(j\omega)$. For many signals, either the phase or the magnitude alone can be used to construct a useful approximation of the signal $y(t)$. For instance, consider the signals $y_2(t)$ and $y_3(t)$ with CTFTs

$$Y_2(j\omega) = |Y(j\omega)| \quad \text{and} \quad Y_3(j\omega) = e^{j\phi(\omega)}.$$

- (c). Demonstrate analytically that $y_2(t)$ and $y_3(t)$ will be real signals whenever $y(t)$ is real.
- (d). Construct a vector `Y2` equal to the magnitude of `Y`. Store in `y2` the inverse Fourier transform of `Y2`. Play this vector using `sound`.
- (e). Construct a vector `Y3` which has the same phase as `Y`, but has magnitude equal to one for each frequency. Store in `y3` the inverse Fourier transform of `Y3`. Play this vector using `sound`.
- (f). Based upon the two signals you just heard, which component of the Fourier transform would you say is most crucial for representing an audio signal: the magnitude or the phase?

These problems consider the effect of scaling the time-axis on the CTFT. Namely, you will examine how the transformation $y(at)$ for $a > 0$ affects the Fourier transform of the signal. For $a > 1$, $y(at)$ corresponds to a compression of the time axis, while $0 < a < 1$ corresponds to an expansion of the time axis. In addition, you will see that if $y(t)$ is sampled sufficiently fast, you can process `y` to obtain the samples you would have obtained by sampling $y(at)$ for $a > 1$. You need not have the continuous-time signal $y(at)$ available to you to do this. You will also see that for $0 < a < 1$, you can process `y` in discrete-time to approximate the samples you would have obtained from $y(at)$.

If $a > 1$ and $y(t)$ is defined on the infinite interval $t \in [-\infty, \infty]$, then $y(at)$ is well defined. However, the vector `y` contains samples of $y(t)$ only over the interval $0 \leq t < N\Delta t$. To derive a vector `ya` from `y` which corresponds to samples of $y(at)$ with $a > 1$, the following two assumptions are made: (i) $y(t)$ is zero outside the interval $0 \leq t < N\Delta t$, and (ii) a is an integer. The second assumption ensures that every a -th sample of `y` will be in `ya`.

- (g). Use `y` to create a vector `y4` which contains the values you would obtain from sampling $y_4(t) = y(2t)$ at 8192 Hz. Note that sampling $y_4(t)$ at 8192 Hz corresponds to sampling $y(t)$ with a new Δt_2 equal to twice the old Δt , i.e., $\Delta t_2 = 2/8192$. The vector `y4` should contain N samples, the second $N/2$ of which are equal to zero.
- (h). Play `y4` using `sound(y4,fs)`. Can you explain the change in pitch by comparing the Fourier transform of `y4` with that of `y`? How does the compression of a signal effect its Fourier transform?

When a was an integer greater than one, you were able to extract the desired samples of $y(at)$ directly from `y`. When $0 < a < 1$, a different problem arises. You must interpolate samples to fill in new values between the samples in `y`. To see the difficulty in interpolating these values, consider the following scenario with $a = 1/2$. Let `y5` contain the values you would have obtained by sampling $y_5(t) = y(t/2)$ at 8192 Hz on the interval $0 \leq t < N\Delta t$. Note that these are the same samples you would get by sampling $y(t)$ twice as fast, i.e., at 16384 Hz. Since `y5` will be twice as long as `y`, you must interpolate the additional samples of `y5` that fall between the samples in `y`. Specifically, you will use a linear interpolator. This will allow you to approximate the desired samples of $y_5(t) = y(t/2)$.

- (i). Create the vector `x` given by

$$x(n) = \begin{cases} y(n/2), & n \text{ even,} \\ 0, & n \text{ odd.} \end{cases}$$

Note that `x` is a vector of length $2*N$. If you are using a limited version of MATLAB, this number of elements in `x` might exceed the maximum allowable vector length. In this case, you should first replace `y` with `y=y(1:4096)` before defining the vector `x`.

- (j). Use the MATLAB `filter` function to perform linear interpolation on `x`. The impulse response for the linear interpolator you want to use here is `h=[1 2 1]/2`. Let `y5` be the result of the interpolation.
- (k). Play `y5` using `sound(y5,fs)`. Can you explain the change in pitch by comparing the Fourier transform of `y5` with that of `y`? Remember that the Fourier transform of `y5` contains $2*N$ samples of $Y_5(j\omega)$ over the interval $-\pi f_s \leq \omega < \pi f_s$. You will need to define a new frequency vector analogous to the one given in Part (a) but with $2N$ points. How does an expansion of $y(t)$ affect the Fourier transform?

2. Amplitude Modulation and the Continuous-Time Fourier Transform

This exercise will explore amplitude modulation of Morse code messages. A simple amplitude modulation system can be described by

$$x(t) = m(t) \cos(2\pi f_0 t), \quad (1)$$

where $m(t)$ is called the message waveform and f_0 is the modulation frequency. The continuous-time Fourier transform (CTFT) of a cosine of frequency f_0 is

$$C(j\omega) = \pi\delta(\omega - 2\pi f_0) + \pi\delta(\omega + 2\pi f_0), \quad (2)$$

which can be confirmed by substituting $C(j\omega)$ into the inverse CTFT synthesis equation to yield

$$\cos(2\pi f_0 t) = \frac{1}{2} \left(e^{j2\pi f_0 t} + e^{-j2\pi f_0 t} \right). \quad (3)$$

Using $C(j\omega)$ and the multiplication property of the CTFT, you can obtain the CTFT of $x(t)$,

$$X(j\omega) = \frac{1}{2}M(j(\omega - 2\pi f_0)) + \frac{1}{2}M(j(\omega + 2\pi f_0)), \quad (4)$$

where $M(j\omega)$ is the CTFT of $m(t)$. Since the CTFT of a sinusoid can be expressed in terms of impulses in the frequency domain, multiplying the signal $m(t)$ by a cosine places copies of $M(j\omega)$ at the modulation frequency.

The remainder of this exercise will involve the signal,

$$x(t) = m_1(t) \cos(2\pi f_1 t) + m_2(t) \cos(2\pi f_2 t) + m_3(t) \sin(2\pi f_1 t), \quad (5)$$

and several parameters that can be loaded into MATLAB from the file `ctftmod.mat`. This file is provided to you with your assignment materials. Extract the contents of the provided folder, ensure your MATLAB current working directory is set to this extracted directory, and type `load ctftmod.mat` to load the required data. If the file has been successfully loaded, then typing `who` should produce the following result:

```
>> who
```

Your variables are:

```
af      dash      f1      t
bf      dot       f2      x
```

In addition to the signal $x(t)$, you also have loaded:

- a lowpass filter, whose frequency response can be plotted by `freqs(bf,af)`,
- modulation frequencies `f1` and `f2`,
- two prototype signals `dot` and `dash`,
- a sequence of time samples `t`.

To make this exercise interesting, the signal $x(t)$ contains a simple message. When loading the file, you should have noticed that you have been transformed into Agent 008, the code-breaking sleuth. The last words of the aging Agent 007 were “The future of technology lies in ...” at which point Agent 007 produced a floppy disk and keeled over. The floppy disk contained the MATLAB file `ctftmod.mat`. Your job is to decipher the message encoded in $x(t)$ and complete Agent 007’s prediction.

Here is what is known. The signal $x(t)$ is of the form of Equation (5), where f_1 and f_2 are given by the variables `f1` and `f2`, respectively. It is also known that each of the signals $m_1(t)$, $m_2(t)$ and $m_3(t)$ correspond to a single letter of the alphabet which has been encoded using International Morse Code, as shown in the following table:

A	·—	H	····	O	— — —	V	···—
B	—···	I	··	P	·—·—	W	·— —
C	—···	J	·— — —	Q	— — ·—	X	—···—
D	—··	K	—·—	R	·—·	Y	—·— —
E	·	L	·—··	S	···	Z	— — —·
F	··—·	M	— —	T	—		
G	— — ·	N	—·	U	··—		

- (a). Using the signals `dot` and `dash`, construct the signal that corresponds to the letter ‘Z’ in Morse code, and plot it against `t`. As an example, the letter C is constructed by typing `c = [dash dot dash dot]`. Store your signal $z(t)$ in the vector `z`.
- (b). Plot the frequency response of the filter using `freqs(bf,af)`.
- (c). The signals `dot` and `dash` are each composed of low frequency components such that their Fourier transforms lie roughly within the passband of the lowpass filter. Demonstrate this by filtering each of the two signals using

```
>> ydash=lsim(bf,af,dash,t(1:length(dash)));
>> ydot=lsim(bf,af,dot,t(1:length(dot)));
```

Plot the outputs `ydash` and `ydot` along with the original signals `dash` and `dot`.

- (d). When the signal `dash` is modulated by $\cos(2\pi f_1 t)$, most of the energy in the Fourier transform will move outside the passband of the filter. Create the signal $y(t)$ by executing `y=dash*cos(2*pi*f1*t(1:length(dash)))`. Plot the signal $y(t)$. Also plot the output `yo=lsim(bf,af,y,t)`. Do you get a result that you would have expected?
- (e). Determine analytically the Fourier transform of each of the signals

$$m(t) \cos(2\pi f_1 t) \cos(2\pi f_1 t),$$

$$m(t) \cos(2\pi f_1 t) \sin(2\pi f_1 t),$$

and

$$m(t) \cos(2\pi f_1 t) \cos(2\pi f_2 t),$$

in terms of $M(j\omega)$, the Fourier transform of $m(t)$.

- (f). Using your results from Part (e) and by examining the frequency response of the filter as plotted in Part (b), devise a plan for extracting the signal $m_1(t)$ from $x(t)$. Plot the signal $m_1(t)$ and determine which letter is represented in Morse code by the signal.
- (g). Repeat Part (f) for the signals $m_2(t)$ and $m_3(t)$. Agent 008, where does the future of technology lie?

Final Notes

- All generated figures must include an appropriate title, axis labels, and sufficient explanation within the report.
- Providing only the code output is insufficient; your report must actively analyze the results and explain the methodology used to solve the problems.
- **Academic Integrity:** Refrain from direct copying of code or reports from peers. Any unjustifiable similarities in code or reports will result in academic disciplinary actions.